

LOS Assignment 1

Sanjana Rao U S
261100690016

1. What If There Were No Operating System?

Imagine a computer without an operating system. Identify the problems a user would face and investigate how OS features solve them. Demonstrate at least three features using Linux and present a *‘Without OS vs with OS’* comparison.

An operating system (OS) is the core software that manages a computer's hardware and software resources. It acts as a bridge between a user and the computer hardware.

Problems Without an Operating System

- The computer would not boot properly.
- Every application needs its own code to talk to the hardware directly.
- Only one program could run at a time, with no way to switch between apps.
- There would be no memory protection, so programs could crash into each other's data.
- There would be no user interface at all, meaning no desktop or menus

OS features to solve them

- **Boot Management** helps the computer start properly and load the operating system.
- **Hardware Management** helps applications communicate with hardware without needing to control it directly.
- **Process Management** allows multiple programs to run and lets users switch between them.
- **Memory Management** keeps each program's memory separate and prevents programs from affecting each other.
- **User Interface** provides things like desktops, menus, icons, and buttons to make the computer easy to use.

Without OS vs With OS

Linux Feature	Without OS	With OS
Process Management	Only one program can realistically run at a time as there is no scheduling or context switching	Multiple processes run concurrently via a scheduler using time-slicing and context switching Example: <code>ps</code> , <code>top</code>
Memory Management	Programs must manually track and manage physical memory addresses	OS provides virtual memory thus isolating each process Example: <code>free -h</code> , <code>/proc</code>
File system management	Raw disk blocks with no structure or naming	Organizes file systems with folders, filenames and permissions Example: <code>ls -l</code> , <code>chmod</code>

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

What role do you want to explore in Cybersecurity?

- [Penetration testing](#)
- [Digital forensics](#)
- [Privacy](#)
- [Security Monitoring](#)

1. Penetration testing

Penetration testing (or pen testing) is a safe, authorized simulation of cyberattacks. Security experts use it to find weak spots in a computer system, network, or app. The main goal is to fix these flaws before real hackers can use them to cause harm.

Linux distributions relevant to penetration testing:

Kali Linux

Kali Linux is one of the most used Linux distributions for cybersecurity. It contains many tools for penetration testing, vulnerability analysis, network testing and password security.

Why choose Kali Linux?

Base	Debian
Tool count	600+ pre-installed
Area of use	Network pentesting, wireless attacks, web exploitation, OSCP prep

BlackArch

BlackArch is a lightweight, open-source Linux distribution, built specifically for penetration testers, ethical hackers, and cybersecurity researchers.

Why choose BlackArch?

Base	Arch Linux
Tool count	2,800+
Area of use	Advanced testing, exploit research, malware analysis

2.Digital forensics

Digital forensics is a branch of forensic science that recovers, investigates, and analyzes electronic evidence from digital devices like computers and smartphones. It reconstructs digital events to support cybercrime investigations, legal cases, and corporate security incidents while keeping data intact for court.

Linux distributions relevant to digital forensics:

CAINE

CAINE most commonly refers to [CAINE Linux](#), an open-source Italian Linux live distribution designed and built for digital forensics and incident response.

Why choose CAINE?

Base	Debian
Tool count	100+ forensic tools
Area of use	Disk imaging, evidence analysis, incident response

3.Privacy

Privacy dictates *what* data should be collected, who can use it, and why. Cybersecurity provides the technical tools and defense mechanisms to lock that data away from hackers.

Tails and Qubes solve different problems: Tails protects a single session, Qubes protects ongoing daily use through isolation. Worth running both in the lab.

Tails

Why choose Tail?

Base	Debian (live USB)
Tool count	~10 curated privacy tools
Area of use	One-off anonymous sessions, journalism, whistleblowing, routes traffic through Tor

Qubes OS

Why choose Qubes?

Base	Fedora/Debian
Tool count	Depends on qube,user-installed per VM
Area of use	Daily use with strict task isolation via virtual machines

4.Security Monitoring

Security monitoring is the continuous process of observing a digital environment including networks, servers, endpoints, and applications to detect suspicious behavior, policy violations, or cyber threats in real time.

Security Onion

Why choose Security Onion?

Base	Ubuntu
Tool count	20+ integrated tools (Suricata, Zeek, Elastic Stack, Wazuh)
Area of use	Network monitoring, intrusion detection, threat hunting

3. Exit status: Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using \$? . Create a simple visual/table showing *Command → Result Exit Status → Meaning*. What can a script learn from an exit status?

In Linux, an **exit status** is a number that tells whether a command or program finished successfully or had an error.

1. **0** means the command worked successfully.
2. **Non-zero number** means something went wrong.
3. Linux uses exit status to help other commands know whether the previous command was successful.

Command	Exit Status	Meaning
mkdir test	0	Directory created successfully
ls	0	Lists files successfully
cat file.txt	0	Displays the file successfully
ls abc	2	Failed because the file or folder does not exist
cat abc.txt	1	Failed because the file does not exist

Execution:

1.mkdir test

—(sanjanarao@kali)-[~]

└─\$ **mkdir test**

└─(sanjanarao@kali)-[~]

└─\$ **echo \$?**

0

2. *ls*

```
└─(sanjanarao@kali)-[~]
```

```
└─$ ls
```

```
Desktop  Downloads  Music    Projects  Templates  Videos
Documents  LOS      Pictures  Public    test
```

```
└─(sanjanarao@kali)-[~]
```

```
└─$ echo $?
```

```
0
```

3. *cat fil.txt*

```
└─(sanjanarao@kali)-[~]
```

```
└─$ cat file.txt
```

```
└─(sanjanarao@kali)-[~]
```

```
└─$ echo $?
```

```
0
```

4. *ls abc*

```
└─(sanjanarao@kali)-[~]
```

```
└─$ ls abc
```

```
ls: cannot access 'abc': No such file or directory
```

```
└─(sanjanarao@kali)-[~]
```

```
└─$ echo $?
```

```
2
```

5. *cat abc.txt*

```
└─(sanjanarao@kali)-[~]
```

```
└─$ cat abc.txt
```

```
cat: abc.txt: No such file or directory
```

```
└─(sanjanarao@kali)-[~]
```

```
└─$ echo $?
```

```
1
```